

## SPEC 02 — Base del Bridge y Configuración (Explicación)

Especificación: specs/02-base-bridge-configuracion.md

Estado: Draft | Nivel: Intermedio | Fecha de generación: 2026-07-21

*Aviso: La especificación fuente permanece como el contrato normativo. Esta explicación puede quedar desactualizada.*

### 1. En un minuto

- **Problema:** Para servir datos de uso al dispositivo ESP32 CYD desde la red local sin depender directamente de APIs externas en el microcontrolador, se requiere un servicio "bridge" en el Mac con configuración estricta, aislamiento de red y gestión segura de ciclos de vida. (Especificado)
- **Resultado propuesto:** Crear el componente bridge/ en Python 3.14.6 con FastAPI/Uvicorn, dos listeners independientes (dispositivo en LAN en port 8765 y lifecycle en 127.0.0.1:8766), validación de seguridad en filesystem (Istat, 0700/0600), parser estricto de bridge.env y generación de tokens de 256 bits. (Especificado)
- **Estado actual:** Draft (Borrador). Requiere que SPEC 01 esté Implemented-Verified antes de iniciar su implementación. (Especificado)
- **Beneficiarios:** El sistema completo ai-usage-meter, al disponer de una capa intermedia segura en el Mac sin exponer endpoints administrativos a la red LAN. (Inferido)

### 2. Qué cambia y qué no

#### Observablemente CAMBIA:

- **Componente bridge/:** Se crea un proyecto Python instalable en bridge/ con comandos CLI ai-usage-meter-bridge y ai-usage-meter-init. (Especificado)
- **Configuración de Usuario:** Carga exclusiva de configuración desde ~/Library/Application Support/AIUsageMeter/bridge.env con validación estricta de permisos. (Especificado)
- **Listeners HTTP:** Ejecución coordinada de listener LAN (AI\_METER\_DEVICE\_HOST:8765) sin rutas expuestas y listener local (127.0.0.1:8766) con /healthz y /readyz. (Especificado)
- **Autenticación Interna:** Primitivas internas para generación (256 bits) y comparación segura en tiempo constante (compare\_digest) de tokens de dispositivo. (Especificado)

#### Observablemente NO CAMBIA:

- **Integración con Codex:** No se incluye Codex App Server, lectura ni normalización de métricas de uso (reservado para specs futuras). (Especificado)
- **Rutas de datos de dispositivo:** No se publica /api/v1/usage ni ninguna otra ruta funcional en el puerto 8765. (Especificado)
- **Firmware ESP32:** No se modifica código C++ ni se realiza aprovisionamiento en hardware CYD. (Especificado)

### 3. Antes y después

| Aspecto               | Comportamiento Anterior                                 | Comportamiento Intendido                                                                                      |
|-----------------------|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Arquitectura Bridge   | Componente inexperimento inexistente en el repositorio. | Proyecto Python estructurado en bridge/ con entorno aislado uv y pyproject.toml.                              |
| Configuración Runtime | Sin estándar de almacenamiento ni permisos de archivo.  | Archivo de configuración estricto en ~/Library/.../bridge.env (modo 0600) creado con umask 077.               |
| Servicio HTTP         | Sin servicio bridge activo en el Mac.                   | Dos listeners FastAPI: Dispositivo en LAN (8765) sin rutas y Lifecycle en 127.0.0.1:8766 (/healthz, /readyz). |
| Seguridad de Red      | Sin filtrado de interfaces locales.                     | Validación de IP RFC1918 activa con psutil y rechazo de loopback/públicas/interfaces caídas en LAN.           |
| Manejo de Secretos    | Sin generación ni almacenamiento seguro de tokens.      | Generación de token de 256 bits (SecretStr), logs sanitizados JSON a stderr sin token expuesto.               |

### 4. Cómo encajan las piezas

**Sistema de Archivos y Configuración:** Configuración persistida ~/Library/Application Support/AIUsageMeter/bridge.env leída mediante un parser estricto que valida permisos (Istat 0700/0600) y construye BridgeSettings inmutable con Pydantic.

**Listeners HTTP y Supervisor:** Dos aplicaciones FastAPI/Uvicorn coordinadas: Listener de Dispositivo (escucha en IP RFC1918:8765) y Listener Lifecycle (escucha solo en 127.0.0.1:8766). Un supervisor asíncrono gestiona el estado RuntimePhase y apaga ambos listeners si uno falla.

**Primitivas de Autenticación:** Modulo de seguridad que genera tokens con secrets.token\_urlsafe(32) y compara identidades con secrets.compare\_digest para evitar ataques de tiempo.

### 5. Paso a paso del plan (15 Pasos de Implementación)

#### Paso 1: Crear el proyecto Python reproducible

- Acción: Estructuración de bridge/ con pyproject.toml, uv.lock y Python 3.14.6.
- Propósito: Establecer el paquete base y herramientas de calidad (pytest, ruff, mypy).
- Dependencias/Impacto: Pasos 2 al 15. | Complejidad: Baja | Razonamiento: Bajo

#### Paso 2: Implementar parser estricto de bridge.env

- Acción: Parser UTF-8 sin bibliotecas externas flexibles para procesar CLAVE=VALOR.
- Propósito: Rechazar comentarios inline, comillas, multilinea o variables no autorizadas.
- Dependencias/Impacto: Paso 3. | Complejidad: Media | Razonamiento: Medio

#### Paso 3: Implementar modelo Pydantic inmutable

- Acción: BridgeSettings derivado de BaseSettings restringiendo fuentes exclusivamente a init\_settings.
- Propósito: Prevenir que variables de entorno del proceso o .env del repo alteren la configuración.
- Dependencias/Impacto: Pasos 4, 11 y 12. | Complejidad: Media | Razonamiento: Alto

#### Paso 4: Validar de forma segura la ruta runtime

- Acción: Uso de Istat sobre ~/Library/.../bridge.env verificando UID, permisos 0700/0600 y ausencia de symlinks.
- Propósito: Proteger la lectura del archivo contra ataques de carrera o symlinks maliciosos.
- Dependencias/Impacto: Paso 5 y 11. | Complejidad: Alta | Razonamiento: Alto

**Paso 5: Implementar inicializador atómico e idempotente**

- Acción: Comando ai-usage-meter-init con umask 077, creación exclusiva y token de 256 bits.
- Propósito: Generar bridge.env sin exponer el token en stdout/stderr/logs ni sobrescribir archivos seguros.
- Dependencias/Impacto: Paso 14. | Complejidad: Alta | Razonamiento: Alto

**Paso 6: Implementar política de interfaces de red**

- Acción: Validación con psutil de IP RFC1918 activa (isup=True) en interfaz local.
- Propósito: Evitar bind en interfaces inactivas, loopback, públicas o broadcast.
- Dependencias/Impacto: Pasos 11 y 12. | Complejidad: Media | Razonamiento: Alto

**Paso 7: Implementar primitivas internas de autenticación**

- Acción: Funciones para generar tokens y comparar con secrets.compare\_digest.
- Propósito: Proporcionar una frontera de seguridad robusta para specs futuras.
- Dependencias/Impacto: SPEC 05. | Complejidad: Media | Razonamiento: Alto

**Paso 8: Implementar logging JSON y redacción**

- Acción: Emisión de eventos JSON sanitizados a stderr con timestamp UTC y request\_id interno.
- Propósito: Prevenir filtración de tokens, IPs o tracebacks en logs de producción.
- Dependencias/Impacto: Pasos 9 a 14. | Complejidad: Media | Razonamiento: Alto

**Paso 9: Crear aplicación lifecycle y handlers estables**

- Acción: FastAPI en 127.0.0.1:8766 con GET /healthz y GET /readyz, sin OpenAPI/Swagger.
- Propósito: Ofrecer endpoints de salud internos con respuestas JSON genéricas.
- Dependencias/Impacto: Pasos 11 y 12. | Complejidad: Media | Razonamiento: Medio

**Paso 10: Crear aplicación vacía del listener de dispositivo**

- Acción: FastAPI independiente en IP LAN:8765 sin rutas funcionales.
- Propósito: Asegurar que cualquier petición reciba 404 {"detail": "not\_found"}.
- Dependencias/Impacto: Paso 11. | Complejidad: Baja | Razonamiento: Bajo

**Paso 11: Implementar arranque atómico de ambos listeners**

- Acción: Enlazar sockets autoritativos antes de iniciar servidores Uvicorn.
- Propósito: Garantizar que ambos listeners inicien juntos o ninguno quede abierto.
- Dependencias/Impacto: Pasos 12 y 13. | Complejidad: Alta | Razonamiento: Alto

**Paso 12: Conectar readiness calculada bajo demanda**

- Acción: Evaluación dinámica de /readyz basada en fase, salud de tareas e interfaz de red.
- Propósito: Devolver 503 inmediatamente si cae la interfaz o falla un listener.
- Dependencias/Impacto: Paso 13. | Complejidad: Media | Razonamiento: Alto

**Paso 13: Implementar supervisión de fallos y shutdown coordinado**

- Acción: Manejador de señales SIGINT/SIGTERM y supervisor que cancela y recolecta tareas.
- Propósito: Garantizar apagado limpio sin tareas huérfanas ni tracebacks.
- Dependencias/Impacto: Paso 14. | Complejidad: Alta | Razonamiento: Alto

**Paso 14: Publicar entry points y códigos de salida**

- Acción: Comandos ai-usage-meter-bridge y ai-usage-meter-init con códigos de error 0 a 4.
- Propósito: Proporcionar interfaz CLI clara y determinista.
- Dependencias/Impacto: Paso 15. | Complejidad: Media | Razonamiento: Medio

**Paso 15: Añadir documentación y prueba operativa separada**

- Acción: bridge/.env.example con valores falsos y suite tests/operational separada.
- Propósito: Verificar bind real en Mac y asegurar calidad con ruff, mypy y pytest.
- Dependencias/Impacto: Cierre de SPEC 02. | Complejidad: Media | Razonamiento: Medio

## 6. Decisiones tomadas y compromisos (Tradeoffs)

- **Uso exclusivo de bridge.env:** Desactiva variables de entorno del proceso y archivos .env locales para evitar inyecciones no deseadas de configuración. (Especificado)
- **Validación con Istat (0700/0600):** Rechaza symlinks y exige permisos estrictos para prevenir ataques de sustitución de archivos en el sistema local. (Especificado)
- **Dos listeners Uvicorn aislados:** Aísla las rutas de administración lifecycle (127.0.0.1:8766) de la interfaz de red LAN (port 8765). (Especificado)
- **Readiness evaluado bajo demanda:** No usa polling en segundo plano; calcula el estado en cada llamada a /readyz comprobando tareas e interfaz. (Especificado)

## 7. Dónde debe participar el usuario

- **Ejecución del inicializador:** Ejecutar ai-usage-meter-init --device-host <IP\_LOCAL> --device-id <ID> para generar la configuración por primera vez. (Especificado)
- **Asignación de IP fija local:** Asegurarse de que el Mac disponga de una IPv4 RFC1918 asignada a una interfaz activa antes de arrancar el bridge. (Especificado)
- **Reparación manual de permisos:** Si el bridge falla con código 3 por permisos incorrectos, el operador debe ajustar manualmente la carpeta a 0700 y el archivo a 0600. (Especificado)

## 8. Riesgos y comportamiento degradado

- **Symlink o permisos inseguros en bridge.env:** El bridge se detiene en el arranque con código de salida 3 antes de abrir puertos HTTP. (Especificado)
- **Caída o desconexión de la interfaz IP LAN:** /readyz responde 503 not\_ready mientras /healthz se mantiene en 200 ok. (Especificado)
- **Fallo inesperado de un listener Uvicorn:** El supervisor detecta la caída, marca la fase como FAILED e inicia el apagado completo del proceso. (Especificado)

## 9. Cómo se demostrará el completamiento

- **Verificación de Calidad:** uv run --locked pytest, ruff check, ruff format y mypy terminan con código 0. (Especificado)
- **Creación de Configuración:** ai-usage-meter-init crea ~/Library/.../bridge.env con permisos 0600 y token de 256 bits. (Especificado)
- **Listeners Operativos:** GET http://127.0.0.1:8766/readyz retorna 200 {"status":"ready"} y el puerto 8765 responde 404 a peticiones LAN. (Especificado)
- **Shutdown Limpio:** Enviar SIGINT apaga ambos servidores sin tracebacks en consola y con código de salida 0. (Especificado)

## 10. Glosario

- **Bridge:** Servicio intermediario en Python ejecutado en el Mac que gestiona comunicaciones seguras entre la red local y los servicios de uso. (Especificado)
- **RFC1918:** Estándar de direcciones IP privadas reservadas para redes locales (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16). (Especificado)
- **Istat:** Llamada al sistema de archivos que obtiene información de un nodo sin seguir enlaces simbólicos (symlinks). (Especificado)

- **Uvicorn:** Servidor web ASGI de alto rendimiento utilizado para ejecutar aplicaciones FastAPI en Python. (Observado)
- **Readiness / Healthz:** Endpoints de diagnóstico: healthz indica que el proceso está vivo y readyz indica si está listo para recibir tráfico. (Especificado)

## 11. Lagunas y dictamen final

**Gaps o Lagunas:** El SPEC 02 está en estado Draft. Su contrato técnico está completamente detallado, pero tiene como requisito previo obligatorio que SPEC 01 pase al estado Implemented-Verified antes de iniciar la implementación del bridge.

**Dictamen Final: Needs clarification before approval (SPEC 02 está en Draft y requiere SPEC 01 verificado)**